MAPSM: Mobility-Aware Proactive Service Migration Framework for Mobile-Edge Computing in Consumer Internet of Vehicles

Xuhui Zhao, Yan Shi, Shanzhi Chen, *Fellow, IEEE*, Jianghui Liu, Baofeng Ji, and Shahid Mumtaz, *Senior Member, IEEE*

***Abstract*—Mobile edge computing is considered as a key technology for consumer Internet of Vehicles networks, which provides low-latency, high-reliability network services for end-users. Service migration need to address where to migrate and how to implement service migration procedure based on user mobility. The existing reactive migrating solutions lead to overlong service migration time and end to end latency. The proactive service migration method obtains the target server in advance through mobility prediction, and the service migration procedure starts before communication handover. Based on the above observation, a Mobility Aware Proactive edge Service Migration framework (MAPSM) is proposed in this paper. MAPSM includes the key aspects: 1) predicting the next location of an end user based on ensemble learning method combining recurrent neural networks and geographical embedding Markov chain predictors; 2) using the mobility prediction result to determine the target edge server of service migration, a proactive migration-handover coordinated method is proposed by performing container pre-migration, memory state migration and communication handover. The time planning scheme in the procedure is also designed. Experimental results demonstrate that MAPSM can greatly improve migration performance, effectively reduce end-to-end latency and significantly reduce service migration time. MAPSM outperforms other baseline service migration approaches.**

Received 30 October 2024; revised 7 April 2025; accepted 17 April 2025. Date of publication 23 April 2025; date of current version 14 August 2025. This work was supported in part by the Beijing Natural Science Foundation under Grant L202018; in part by the National Science Foundation Project in China under Grant 61931005; in part by the Key Research and Development Projects in Henan Province under Grant 241111221700; and in part by the International Cooperation Projects in Henan Province under Grant 232102521005. *(Corresponding author: Yan Shi.)*

Xuhui Zhao is with the College of Information Engineering, Henan University of Science and Technology, Luoyang 471023, China, and also with the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: zxh@haust.edu.cn).

Yan Shi is with the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing 100088, China (e-mail: shiyan@bupt.edu.cn).

Shanzhi Chen is with the National Engineering Research Center of Mobile Communications and Vehicular Networks and the State Key Laboratory of Wireless Mobile Communications, China Academy of Telecommunication Technology, Beijing 100191, China (e-mail: chensz@cict.com).

Jianghui Liu and Baofeng Ji are with the Computer Department, Henan University of Science and Technology, Luoyang 471023, China (e-mail: hl1211_hui@163.com; fengbaoji@126.com).

Shahid Mumtaz is with the Department of Applied Informatics, Silesian University of Technology, 44-100 Gliwice, Poland, and also with the Department of Engineering, Nottingham Trent University, NG1 4FQ Nottingham, U.K. (e-mail: dr.shahid.mumtaz@ieee.org).

Digital Object Identifier 10.1109/TCE.2025.3563627

***Index Terms*—Proactive service migration, ensemble learning, mobile edge computing, container migration.**

I. Introduction

MOBILE Edge Computing (MEC) is considered a key technology in consumer Internet of Vehicles (IoV) networks, which allows real-time processing of large amounts of data at the network edge [1]. The similar concept is called Fog computing [2]. Considering the strict time requirements of new services in consumer IoV networks, such as online gaming, on demand streaming and assisted driving services, the increased mobile data traffic can be handled with MEC. As an enabler for these new service in IoV networks, MEC deploys computation, storage, and communication resources closer to end-users (vehicles, or mobile users in vehicles). MEC allows end-users to offload its computation-intensive applications to nearby edge cloud server to reduce end to end delay. In MEC, an edge cloud server is located at wireless access points such as base stations, or roadside unit and hosts virilized environment such as containers. Note that not all wireless access points are collocated with an edge server and the coverage of a single edge server is limited.

The end users' mobility is common in MEC system. When an arbitrary user moves outside the service area of the source edge server, the ongoing edge services may suffer from interruption or dramatic drop in quality of service. Service migration has great potential to address the issues [3], [4], [5], [6], which is assumed as an enabler of realizing intelligent mobility sensing in MEC. Service migration strategy should be made when an arbitrary end-users moves outside the service area of the associated edge cloud server, including that where and how to migrate the ongoing edge services. Service migration involves the following challenges.

Firstly, massive and diverse IoV services are running on MEC networks, and the services are also becoming much more time-sensitive. Service migration involves hundreds of files, migrating their replicas on every nearby node to follow end users' mobility reactively is undesirable, given the typical limitations in storage and network capacity. Therefore, where to migrate the running service should be carefully decided.

Secondly, the procedure of service migration involves not only the migration of massive container files (including base container image copy and application copy, and memory state

1558-4127 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:08:44 UTC from IEEE Xplore. Restrictions apply.

copy) but also the wireless connection handover in communication aspect. Reactive container migration follows the communication handover, which means that the migration of massive container files is after communication handover, leading to an extension in the service migration time. Therefore, how to implement proactively service migration should be carefully planned to achieve minimal migration time.

In MEC, service migration and mobility prediction are closely interconnected. Mobility prediction analyzes users' historical trajectories and behaviors to predict their future locations and mobility paths, thereby providing decision-making support for service migration. By proactively migrating services to edge nodes closer to users' next locations, the system can reduce latency, optimize resource utilization, and ensure service continuity. The integration of these two aspects enhances the efficiency and user experience in MEC.

Despite many researches have focused on mobility-induced service migration in the edge computing environment, migration strategy design in these research only considered whether or where to migrate [7], and migration path selection [8], [9]. Most work utilized the hypothetical or random mobility model or assumed the end-users'trace is known in advance [10], [11], [12], [13], [14], and exploited reactive approaches in which the service is migrated to follow the users' mobility [15], [16], [17]. Under this circumstance, containers or virtual machine needed to be migrated after the users have moved into the coverage of new edge cloud. Only a few researches considered jointly handover and migration process [14], [18], [19]. The proactive approaches have been adopted in [20], [21] using mobility prediction. This proactive approach requires to accurately predict where and when end-users are likely to move, and to migrate the running service from end-users ahead of the handover taking place. However, these works assumed that the service on target edge servers are available or use virtual machine as the virtualized environment [10], [19], [20], [22], [23], [24].

The accurate mobility prediction model is important for implementing proactive service migration. Using deep learning to predict the next location is a hot research topic. The existing research use attention recurrent network, deep learning network model based on full connection and layered gated recurrent unit, long short term memory network, and convolutional neural network to predict the next location of end-users (mobile users or vehicles) [26], [27], [28], [29]. In these research, mobility prediction methods are proposed, but the accuracy needs to be further improved. In order to improve the accuracy of the prediction model and reduce the risk of over fitting, the ensemble learning method combines multiple weak learners to obtain better and more comprehensive strong learners [30], [31]. Thus this paper adopts an ensemble learning method combining recurrent neural network and preference embedded Markov chain to improve the accuracy and generalization performance of the mobility prediction model.

In this paper, we jointly consider the migration decision (where to migrate) and migration procedure, adopt ensemble learning method to predict user mobility and propose an proactive service migration method based on containers to optimize service migration time and end-to-end delay. This container migration method includes the pre-migration of container images and application images, memory state data migration, and communication handover. The main contributions are as follows:

(1) We present a MAPSM framework that migrate proactively container and implement communication handover for end-users based on mobility prediction. Ensemble Learning-based Mobility Prediction (ELMP) is proposed, which combines recurrent neural networks and geographical Preference Embedding Markov Chain (PEMC), returns prediction results in the form of a probability distribution over the next location which the edge servers are deployed, and mobile users may handover toward the edge server.

(2) We propose a Proactive Migration-Handover Method (PMHM) to minimize end to end delay and total migration time. The method include two aspects: (i) where to migrate, which determines the target edge server and the base station for handover based on the result predicted by ELMP and (ii) the time when to perform various stages of the migration process and the handover.

(3) ELMP is evaluated using a real-life mobility dataset. The experimental results demonstrate that the proposed method has outperformed other existing approaches in accuracy. The simulation results of PMHM also demonstrate that the proposed framework MAPSM outperforms the baseline service migration approach.

The rest of the paper is organized as follows. The related work is reviewed in Section II. The system model is described in Section III. In Section IV, problem formulation is described. In Section V, the ensemble learning-based mobility prediction method ELMP is put forward. In Section VI, the Proactive Migration-Handover Mechanism PMHM is presented. Experiments Results are shown in Section VII. The conclusion of the paper and future directions are drawn in Section VIII.

II. Related Work

Mobility is an important feature in MEC in consumer IoV, which poses a critical challenge for maintaining service continuity. Service migration has been considered as a promising technology, especially for the computation-intensive and latency-sensitive applications. The service migration in MEC not only involves communication handover but also container or virtual machine migration based on different virtualized technologies. In this paper, we mainly focus on container migration.

A. Service Migration

Service migration is induced by user mobility or load balancing [7], and where to migrate and how to implement migration process is determined. The majority existing works [10], [16] focus on the reactive service migration triggered by communication handover. When mobile users leave the coverage area of the source edge server, the offloaded service is migrated from the source edge server to the target edge server. This reactive method only applied to the delay

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:08:44 UTC from IEEE Xplore. Restrictions apply.

tolerant services. However, for the delay-sensitive services, the proactive service migration is a better solution [20], [21]. Proactive service migration can be made by mobility prediction. The currently proposed proactive solutions rely on virtual machine, and consideration about user mobility mainly use traditional random mobility, such as random waypoint mobility model or moving speed [6], [10], [11], [14], [19]. Moreover, the problem formulation entails a strict optimization task that is difficult to deploy in practice. In our work, we aim to improve the accuracy and adaptivity of mobility predictions algorithm using ELMP. ELMP can adapt to the new mobility dataset and obtain better prediction performance. Furthermore, we enhance service migration method proposed by [15], by optimizing the end to end delay and total migration time using PMHM, by performing container pre-migration, migration and paralleled communication handover.

B. User Mobility Prediction

The accurate and adaptive mobility prediction models are key to the design of service migration strategies. The current works focus on deep learning for the next location prediction models. The authors in [25] propose an Long Short Term Memory (LSTM)-based mobility prediction mechanism to predict the probable next position of a mobile device. The authors in [26] predict an individual's next location as well as when he will visit it, which adopt deep learning model based on fully connected and hierarchical Gated Recurrent Units (GRU). Reference [27] is based on an RNN, which provided a good basis for our further research. The reference Using historical/real-time travel trajectories, a DeepTrip model is proposed based on a trip sequence-to-sequence deep learning structure coupled with an attention mechanism [28]. Reference [29] assessed the effectiveness of various neural network models (CNN, RNN, LSTM) in predicting coordinates for three mobility models.

The above research provides new methods for mobility prediction. To improve the accuracy of prediction models while reducing the risk of overfitting, the ensemble learning method is proposed in [32], [33]. Ensemble learning is to combine multiple weak learners in order to get a better and more comprehensive strong learners. Based on the above research, we propose the ELMP which combining RNN and GEMC using soft weighted average ensemble method, for improving accuracy and generalization performances of mobility prediction model.

C. Virtual Machine/Container Migration

Virtual Machine (VM) and container are two virtualized technologies used in MEC. VM migration has been researched in many existing works. The main difference between VM and container is that VM is installed on system hardware, and relies on the VM manager to manage system resources. However, containers are implemented through operating system virtualization and share the underlying operating system kernel. From the basic principles of these two virtualization technologies, containers reduce the operating systems layer in the client, making the system more lightweight and

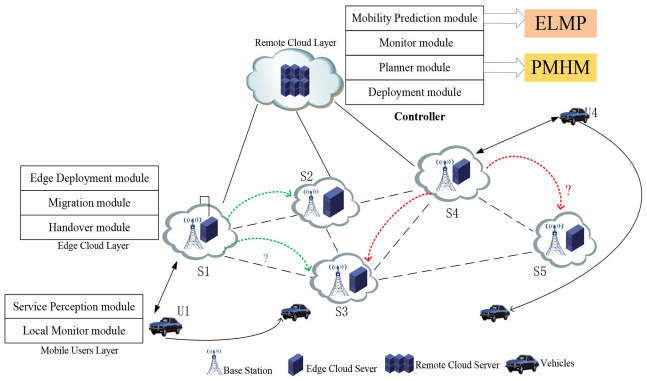


Fig. 1. MAPSM Framework.

having high-performance. Thus, containers are more suitable for the time sensitive application in service migration. Furthermore, the process design of VM/container migration methods involves storage data migration (hard disk data) and memory data migration. The migration of storage data includes the basic image files of containers or VMs, while the migration of memory data requires migrating all status data about services (CPU status, memory data, and devices data, etc.) to the target server. However, due to the fact that VM image files are larger than container image files, and the iterative copying of memory data files in VM migration generates a large amount of file transfers, thus VM cause excessive service migration time and downtime.

However, most of the work focus on virtual machine migration over WANs which is more relevant to cloud computing [34], [35]. These research results are not applicable to MEC environment. Some work in MEC environment considered jointly virtual machine migration and communication handover [10], [14], [18], [19]. Recently, containers are gaining popularity due to its low virtualization overhead and fast responsiveness [5], [36], [37]. This has attracted a large amount of work proposing novel solutions in container-based service migration [18], [36], [38], [39]. However, they assume that the service (container base image and application replicas) on target edge servers are available. In our work, we consider a comprehensive framework, including both target edge server selection and migration of the offloaded service, together with communication handover for end-users. In addition, we evaluated ELMP by using a real-life mobility dataset, and we jointly minimize end to end delay and total migration time by optimizing various stages of the migration process.

III. System Model

We consider a hierarchical framework, which consists of end-users layer, edge servers layers, and central server layer. As depicted in Fig. 1, in the end-users layer, a vehicle which can move from one location to another. In the edge server layer, the edge servers receive offloading requests from end users and provide computation-intensive service. These edge servers are collocated with Base Station (BS) at the access network and have limited coverage area. Due to end users' mobility, the offloaded service need be migrated and executed communication handover to guarantee service continuity. In

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:08:44 UTC from IEEE Xplore. Restrictions apply.

the top layer, the central server hosts one or more servers or containers that have all the service copies. The central server collects and stores mobility trace data, specific location information in a central database. The controller can be placed on central server, and consists of the four parts, which are Mobility Prediction module, Mobility Prediction module, Monitor module, Planner module. Each module is described as follows:

1) a Mobility Prediction module is placed, which combines Recurrent Neural Networks (RNN) and Geographical Embedding Markov Chain (GEMC) predictors to generate high-accuracy estimation of next location in the near future.

2) Mobility Prediction module receives offloading requests from end users, notifies the Planner module to find an edge server for placing offloaded services, and sends this instruction to the corresponding edge server.

3) the Monitor module monitors the service performance such as the end to end delay and the Received Signal Strength Indication (RSSI). If the service performance is below a certain threshold, the monitor module inform mobility prediction module to perform mobility prediction algorithm ELMP. The Mobility Prediction module sends the next location prediction result to the Planner module.

4) The Planner module makes the service migration scheme using PMHM, including where and when to migrate and handover.

Each MEC server consist of the following parts: 1) Edge-Deployment module deploy multiple containers according to instructions from central controller to execute the service requests from end users. 2) The Migration module on the source and destination edge servers collaborate with each other to migrate containers based on the instructions from Planner Module in the central controller.

Whenever a mobile user connect to a BS, the mobile user can send location information (e.g., GPS) to the central server by a fixed time period and use a Service Perception module to request to offloading tasks to the edge servers. The Local Monitor module monitor the End to End (E2E) delay and RSSI information with regard to the nearby BS and edge servers and the monitored information is sent to the central controller.

As shown in Figure 1, in a certain time slot, the end users U1 and U4 are within the coverage of edge servers S1 and S4 respectively. Due to the nature of user mobility, end users U1 and U4 may leave the coverage of S1 and S4 in a future time slot. The controller entity needs to make service migration planning based on the monitoring module and mobility prediction module. To ensure the service continuity, the controller entity needs decide where to migrate the corresponding services from U1 and U4, and how to implement the container migration process. The green line indicates the possible migration target of the services from S1, and the red line indicates the possible migration target of the services from S4.

IV. Problem Formulation

Suppose that mobile user set is $\mathcal{U}$, base station set is $\mathcal{B}$, and edge servers set is $\mathcal{S}$. t denotes a time slot, and t' denotes a future time slot. At time slot t, a mobile user $u \in \mathcal{U}$ who connected to a BS b offloads computation task to edge server s deployed as a container C_u. w^t_{ubs} is a binary variable, which denote whether or not u is connected to the BS b, and its offloaded service is allocated to the server s at time slot t. Let d^t_{ubs} denote the E2E delay which refers to the interval between the time when u send the offloaded task to the edge server s through its associated BS b, and the time when u receive the result after task execution completion.

At a future time slot t', the mobile user leaves the current base station b and moves to another BS b', all the offloaded tasks are migrated to the new edge server s' which is predicted by the ELMP. Suppose that $\hat{n^{t'}_u}$ denote the number of tasks that are offloaded from mobile user u to edge server. Thus, s' is to host the container and the new base station b' is allocated as wireless access point for keeping the wireless connection with the mobile user. $w^t_{ub's'}$ is a binary variable, which denote whether or not u is connected to the BS b', and its offloaded service is allocated to the server s'. As a result of PMHM, the change of the End to End (E2E) delay is $\Delta\hat{d^{t'}} = \hat{d_{ubs}}^t - \hat{d_{ub's'}}^{t'}$. Thus $\forall u \in \mathcal{U}$, the time gain obtained from PMHM is defined as

$$T = \sum_{b=1}^{|\mathcal{B}|}\sum_{s=1}^{|\mathcal{S}|}\sum_{b'=1}^{|\mathcal{B}|}\sum_{s'=1,u\in\mathcal{U}}^{|\mathcal{S}|} \Delta\hat{d^{t'}}\hat{n^{t'}_u}w^t_{ub's'}, \tag{1}$$

Besides the time gain, PMHM will also induce service interruption which is usually measured by the service downtime. Service downtime starts the time when the container C_u serving for mobile user u becomes unavailable and ends when the service becomes available again on the new edge server. Hence we denote it by $\hat{DM_{ubb'ss'}}^{t'}$, and the time cost of PMHM can be denoted by

$$C = \sum_{b'=1}^{|\mathcal{B}|}\sum_{s'=1}^{|\mathcal{S}|} \hat{DM_{ubb'ss'}}w^t_{ub's'}, \tag{2}$$

Our objective is to maximize time profit which is oriented mobile user, defined as follows

$$argmax(T - C), \tag{3}$$

subject to:

$$\sum_{s=1}^{|\mathcal{S}|}\sum_{b=1}^{|\mathcal{B}|} w^t_{ubs} = 1, \tag{4}$$

$$\sum_{s'=1}^{|\mathcal{S}|}\sum_{b'=1}^{|\mathcal{B}|} w^{t'}_{ub's'} = 1, \tag{5}$$

$$w^t_{ubs} = 0 \quad if \quad R^t_{ubs} < R_{min}, \tag{6}$$

$$w^{t'}_{ub's'} = 0 \quad if \quad R^t_{ub's'} < R_{min}, \tag{7}$$

The constraint (4) and (5) means that an application can be only offloaded to a single edge server, and a mobile user can only access a single base station. The constraint (6) and (7) mean that a mobile user will not access a based station if the measured RSSI below the minimum RSSI in the sighed SLA.

Based on the prediction result of PMHM, the problem (3) can be founded a feasible solution, proactive migration plan

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:08:44 UTC from IEEE Xplore. Restrictions apply.

for end users, which means $w^{t'}_{ub's'} = 1$. In this condition, the user u is connected to base station b' and allocated an offloaded service to server s' at the future time t'. In the following parts, we will give how to compute the end to end delay $\Delta\hat{d}^{t'}$, and the service downtime $\hat{DM}_{ubb'ss'}{}^{t'}$.

1) End to End Delay: in general, the E2E delay include the following components. The first component is the processing delay which refer to the time of implementing the offloaded task on the edge server, depending on the computation speed of the edge server. The transmission delay refers to the duration between sending input data of the offloading task and receiving the result of the offloading task. We neglect the delay of receiving the result, as the size of the task result is usually much smaller than the input size of the task itself. At the time slot t, the base station b and the edge server s are the service nodes. At the time slot t', the base station b' and the edge server s' are considered as the new service nodes. Therefore, the E2E delay the time slot t is computed as:

$$\hat{d_{ubs}}{}^{t} = I^{t}_{u}/B^{t}_{ubs} + P_w/P_s + O^{t}_{u}/B_{ubs}, \tag{8}$$

where I^{t}_{u} refers to the input data size of the offloaded task from the end user u at time slot t, O^{t}_{u} refers to the output data size of the offloaded task from the end user u at time slot t. After completing service migration, if the service on edge server s' is available,the E2E delay the time slot t' is computed as:

$$\hat{d_{ub's'}}{}^{t'} = I^{t'}_{u}/B^{t'}_{ub's'} + P_w/P_{s'} + O^{t'}_{u}/B_{ub's'}, \tag{9}$$

If the service migration procedure is not completed, the service on edge server s' is not available, the waiting time is added into E2E delay, which is computed as:

$$\hat{d_{ub's'}}{}^{t'} = T_{wait} + I^{t'}_{u}/B^{t'}_{ub's'} + P_w/P_{s'} + O^{t'}_{u}/B_{ub's'}, \tag{10}$$

where P_s and $P_{s'}$ denote the computation power of the source edge server s and the target edge server s', respectively. P_w denotes the CPU cycles for completing the task. $I^{t'}_{u}$ refers to the input data size of the offloaded task from the end user u at time slot t', $O^{t'}_{u}$ refers to the output data size of the offloaded task from the end user u at time slot t',$B^{t'}_{ubs}$ and $B^{t'}_{ub's'}$ refer to the transmission speed between the mobile user and edge server, which are denoted as

$$B^{t'}_{ubs} = min\left(B^{t'}_{ub}, B^{t'}_{bs}\right), \tag{11}$$

$$B^{t'}_{ub's'} = min\left(B^{t'}_{ub'}, B^{t'}_{b's'}\right), \tag{12}$$

If the service is available on the target edge server, the change of E2E delay after service migration is computed as:

$$\Delta\hat{d}^{t'} = \hat{d_{ubs}}{}^{t} - \hat{d_{ub's'}}{}^{t'}, \tag{13}$$

$$\Delta\hat{d}^{t'} = P_w * (1/P_s - 1/P_{s'}) + I^{t'}_{u}\left(1/B^{t'}_{ubs} - 1/B^{t'}_{ub's'}\right) + O^{t'}_{u}\left(1/B^{t'}_{ubs} - 1/B^{t'}_{ub's'}\right), \tag{14}$$

If the service is not available on the target edge server, the waiting time T_{wait} is added into the change of E2E delay, as shown in equation (10).

Based on our proposed solution PMHM, the user u is connected to base station b' and allocated an offloaded service to server s' at the future time t'. we suppose that $P_{s'}$ is greater than P_s. At the time slot t', the transmission speed (bandwidth) between the mobile user and base station $B^{t'}_{ub'}$ is greater than $B^{t'}_{ub}$, which can be estimated based on the RSSI and the access wireless technology. Thus, we can obtain $B^{t'}_{ubs} < B^{t'}_{ub's'}$, and the change of E2E delay $\Delta\hat{d}^{t'}$ could be positive if the PMHM is made properly.

V. Ensemble Learning-Based Mobility Prediction (ELMP)

In this section, we present the method to model user mobility pattern and predict the next location with high-accuracy estimation in the next time slot. This location is then used within the proposed PMHM optimization strategy for guaranteeing service continuity. Note that the prediction of the next location and the optimization solution for migration plan using PMHM are independent module executed on central cloud by controller entities. At the time the Planner module has to decide where the containers related to the user's service to migrate, the Mobility Prediction module sends the most updated next location vector for each users.

Whenever a mobile user connects to the BS, the central cloud server collect location information (such as GPS) from the mobile user within a fixed period of time period S. This location in the trajectories is collected through latitude and longitude, which describe its relative position with respect to the serving base station. In the following part, we refer to a trajectory as the sequence of the location that describes the positions of the mobile user from the time it connects to a base station ($t = 0$) to the instant in which it hands over to another base station ($t = LS$), where L is the number of samples of the complete trajectory, collected with a sampling period of S seconds. Formally, an L-long trajectory is denoted by $p = \{p_1, p_2, \ldots, p_i, p_{i+1}, \ldots, p_L\}$, with $p_i = (t_i, \alpha_i, \beta_i)$

Next location prediction of end users is a complicated task because of complex sequential transition regularities and multi-level prediction (daily routines, weekend leisure, or personal periodic activities). Moreover, user mobility data is heterogeneity and sparsity in nature, as the location information is recorded only when the user accesses the location service. Thus, we design a Ensemble Learning-based Mobility Prediction (ELMP) algorithm for predicting user mobility. The algorithm is executed by two blocks: (i) the first level learner, consisting of a Recurrent Neural Network (RNN) and a Preference Embedding Markov Chain (PEMC) model (see Sections V-A and V-B), that uses the user mobility data samples as input. (ii) The results from (i) then combined to obtain the final estimates using ensemble learning based on Soft Weighted Average Method (see Section V-C), which is called second level learner. The two blocks are shown in Fig. 2, while the training process is detailed in Algorithm 1.

The benefits of adopting an RNN and PEMC mobility predictor and ensemble learning algorithm are two-fold. The first reason is that this RNN and PEMC model is beneficial to deal with sequential data, and emphasizes the dependencies among sequence location nodes. The second reason is that ensemble learning based on soft weighted average method

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:08:44 UTC from IEEE Xplore. Restrictions apply.

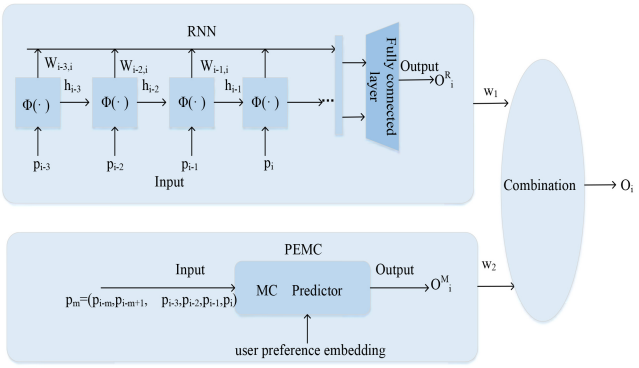


Fig. 2. ELMP Pipeline.

is key to automatically capture complex mobility behaviors, which would be predicted by the learner using new training datasets produced by cross-validation procedure.

A. Recurrent Neural Networks (RNN)

RNN has been widely used in sequential prediction. To deal with the sparsity, temporal periodicity and spatial regularity of user mobility, a RNN architecture incorporating spatiotemporal distances is proposed [27].

Suppose that a user's mobility trace is represented as a sequence $p = \{\ldots, p_{i-3}, p_{i-2}, p_{i-1}, p_i, p_{i+1}, \ldots\}$, the temporal and spatial distances between two check-ins is denoted as $\Delta T_{i,j}$ and $\Delta D_{i,j}$ respectively. The historical hidden states is denoted as h_{i-1},h_{i-2},h_{i-3},..., h_{i-n}. The aggregated hidden state is denoted by $\mathcal{W}_{i,j}$ $(\Delta T_{i,j}, \Delta D_{i,j})$, which can be computed as:

$$\mathcal{W}_{i,j}\big(\Delta T_{i,j}, \Delta D_{i,j}\big) = hvc\big(2\pi \Delta T_{i,j}\big) e^{-\alpha \Delta T_{i,j}} e^{-\beta \Delta T_{i,j}}, \quad (15)$$

where α is a temporal decay rate, β is a spatial decay rate, $hvc(x) = \frac{2}{1+\cos(x)}$ is the Havercosine function modeling the daily periodicity. Furthermore, a learnable user embedding vector for each user, which is concatenated with the aggregated hidden state and then fed into a fully connected layer for predicting the next location. The output of the last fully connected layer with SoftMax activation function that produces an l-dimensional vector, $O_i^R = (O_{i,1}^{RNN}, O_{i,2}^{RNN}, \ldots, O_{i,l}^{RNN})$, containing the probability that the user will move to each of neighboring cells location after leaving the current location.

In our method, we use vanilla RNN units with 10 neurons per layer, and a full connected layer with a learning rate of 0.01. Each user traces is input as input sequence, we computed weights in hidden state layers and normalized the weights, then we applies the weighted average using spatial and temporal data in combination of user embeddings to the output of a RNN unit. Note that the historical hidden state in multi-step of RNN is used for cancelling the noise, capturing the highest predictive power. Thus it is important to decide how many past hidden states should be considered. In practice, the empirical analysis shows that this number depends on the average time between successive check-ins.

Algorithm 1 Ensemble Learning-Based Mobility Prediction Algorithm (ELMP)

Input data management parameters: *sequencelength*, *batch_size*
Input RNN parameter: *learning_rat*, *epochs*, *hiddenlayer*, *weightdecay*; *validateepoch*
Input PEMC parameter: γ, τ

Add user location data D; ▷*Create RNN trainer*
Instantiates RNN with spacial weight α and temporal weight β;
Set Adam optimizer;
Compute O_i^S, O_i^G
Output probability vector O_i^M according to equation (19)
for $t = 1, 2, ...,$ *epochs* **do** ▷*TrainingLoop*
    Shuffle users;
    for $j = 1, 2, ...,$ *enumberate*(*datalocader*) **do**
        for all batches **do**
            Forward propagate the input;
            Compute loss based on cross entropy loss function;
            Backpropagate the error through network;
        end for
    end for
end for
Compute ensembled vector O_i according to equation (20)
Evaluate prediction performance with validation set;
Compute the prediction accuracy of validation set *acc*
acc ⟵ accuracy on the validation set;
if *acc* > *maxacc* **then**
    save network parameters(weights,bias,w_1,w_2);
    maxval ⟵ *acc*
else
    if *random*(0, 1) > ϵ **then**
        Restore previously saved network parameters;
    end if
end if
Output vector O_i^R
return :O_i

B. Geographical Preference Embedding Markov Chain (PEMC)

Since user mobility behaviors are influenced by many factors such as user preferences, geographical distance, friendship and so on. Considers only one factor, which does not reflect the user-specific transition. We further develop a personalized geographical preference Embedding Markov Chain (PEMC) method, which considers the sequential information and individual geographical preference together. We utilize two latent metric: one is the times the sequence transition and the other is the user preference.

Users' mobility behavior usually follow certain sequential patterns. For example, a user likes to go to coffee shop after lunch; therefore it would be logical to infer her next location as the coffee shop. Order-m Markov model considers the latest m locations of a user, and searches for the most frequent

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:08:44 UTC from IEEE Xplore. Restrictions apply.

patterns to predict the next location. The value of m used in this work is set according to the dataset in the experiments. Therefore, we formulate the sequence transition information as follows: let $p_{i,j}$ denote j neighbouring location sequence to the current p_i, $1 < j < l$, l denote the maximum number of candidate edge servers. We utilized $p_{i-m,m}$ represent the location sequence visited up to time slot t, and $p_{i-m,m} = (p_{i-m}, p_{i-m+1}, \ldots, p_i)$. Each element of the l-dimensional vector O_i^S can be computed as:

$$O_i^S = \left(O_{i,1}^{SMC}, O_{i,2}^{SMC}, \ldots, O_{i,l}^{SMC}\right), \tag{16}$$

$$O_{i,j}^{SMC} = \frac{\sum_{(p_m, p_{i+1,j}) \in D} Num\left(p_m, p_{i+1,j}\right)}{\sum_{p_m \in D} Num(p_m)}, \tag{17}$$

where D denotes the whole dataset, $Num()$ refers to the times that the sequence inside the bracket appears in the training dataset.

Users would like to visit the near locations rather than the far way locations. Geographical distance affects users mobility behavior. Some previous works [41] have studied the probability of location visiting, i.e., the travel distance, and found that it is inversely proportional. Based on the above finding, we propose to incorporate the geographical preference into the Markov chain model. Each user u has a latent position $O_{u,i}$, which denotes the Euclidean distance of user u and location i in the user preference space. We assumed that the user preference space satisfy Normal distribution. The user preference model is defined as $O_i^G = O_{i,1}^G, O_{i,2}^G, \ldots, O_{i,l}^G$, each element is defined as below.

$$O_{i,j}^G = \left(\gamma O_{u,i} + (1-\gamma) O_{i,j}^S\right), \tag{18}$$

where γ controls the weight of different kinds of factors influencing user mobility. The output of PEMC is defined as below.

$$O_i^M = \begin{cases} O_i^S & \Delta T_{i,j} < \tau, \\ O_i^G & otherwise, \end{cases} \tag{19}$$

where $\Delta T_{i,j}$ denotes the time difference of two successive checkins p_i and p_j. τ denotes the time interval threshold between two adjacent check-ins. $\Delta T_{i,j} < \tau$, it indicates that successive check-ins have small time difference, which may indicate their strong relevance. In this condition, only the sequence pattern information is considered. Otherwise, the preference factor is considered.

C. Ensemble Learning Based on Soft Weighted Average Method

In the previous Sections V-A and V-B, we described the RNN and PEMC model in detail, which is used as base models. To improve prediction accuracy and reduce overfitting, Soft Weighted Average Method is adopted to ensemble the learning results of every based model. The final probability vector $O_i = (O_{i,1}, O_{i,2}, \ldots, O_{i,l})$ is ensemble by using weights as follows:

$$O_i = w_1 O_i^R + w_2 O_i^M, \tag{20}$$

where w_1 and w_2 indicate the weight given to each learner and satisfies $w_1 + w_2 = 1$.

This ensemble method allows the RNN and PEMC to have different weights. This is because the two methods described above are trained for the same data set. Meanwhile both of the methods introduce the temporal and spatial factors into the prediction model. However, the prediction ability of the two models has certain differences, the prediction results need to be weighted for obtaining an ensemble model with superior prediction and generalization performances.

D. Data Processing

The data is split in batches using a fixed length sequence of locations according to user identifier. An active user has at least one sequence in the batch. If the location sequence length of an active user is less than the fixed length, then its location sequence is replaced with a new user's location sequence. If there are no unused available, the processed ones are reused. The batch guarantees that each location sequence of each user was processed at least once. If a user's location sequence length is greater than the fixed length, the latest checkins are removed. The dataset are spilt into 80%/20% for training/test data, where the test data are strictly after the training data. To obtain at least one test location sequence with label, we choose the users who have at least 5*sequence length+1 checkins in total. Every data record is one checkin stored in the following format, userid, timestamp, latitude, longitude, locationid. The checkins for the same user have to be on continuous lines. Therefore, we recreated the identifiers for users and locations, which is continuous from 0.

E. Training Process

In this part, we describe how the solution are intergraded and trained, which is shown in Algorithm 1. First, the training dataset are loaded into RNN model. Second, RNN trainer is created. In RNN trainer, a batch of user location sequences are forwarded through RNN model to obtain the probability vector O_i^R for the next location. Next, training loop is implemented and the Adam stochastic optimizer with cross entropy loss is used for backprogagation. This enables the RNN network to become more accurate as more user samples is input into the network. Every 100 epochs, the RNN model is evaluated using the test data. The integration parameters w_1 and w_2 are updated and saved if they obtain higher accuracy than the previous ones. We use ϵ greedy approach to decide whether the training process continue or restart from the previous network parameters. During each training round, the ϵ greedy and ensemble method increase the generalization capacity of the framework while more user samples are input.

VI. Proactive Migration-Handover Mechanism (PMHM)

Container is with layered storage, which include base image, application image, and memory state image, enabling fast packaging and migration between source edge server and target edge server. As mentioned before, by mobility prediction, the target server collocated with the next location where the user moves can be obtained in advance. The target server can obtain the basic image file and application image

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:08:44 UTC from IEEE Xplore. Restrictions apply.

file before service migration, so the transmission time for the basic image file and application image file can be reduced in the actual migration process.

A. The Procedure of PMHM

Based on the results predicted by ELMP and container migration technology, we propose a Proactive Migration-Handover Method (PMHM) to minimize the service migration time and E2E delay. The method include two aspects. Firstly, the data in migration procedure includes base image and application image, which is to be pre-migrated to the target edge servers and the BS for handover. Secondly, the best triggering time, which determines the coordinated time to trigger various stages of the migration process and the handover process. In this paper, we select the edge server with highest probability in the results predicted by ELMP as the target edge server, which is represented as s'. Checkpoint/Restore In Userspace (CRIU) is utilized to perform container migration for stateful application. The procedure is designed as follows:

1) Disk migration: The end users are in the coverage of the source base station b and edge server s at time slot t. In this phase, the disk data including base container image and the application data image are copied from the source edge server s to target edge server s'. Then a cloning process (base image and application) on the target edge server s' is started, which is followed by the pre-copy memory state migration.

2) Memory state migration:

a. In this phase, we pre-dump (i.e., snapshot the memory of) the current container on source edge server s and transfer the whole memory state to the target server s', while leaving the application from the container on source edge server s continue to run.

b. Delta memory state migration: at time slot t', the mobile user enter into the coverage of the base station b, the communication handover to b' is performed. In this phase, we dump the container and save the changes between this and the previous dump as a delta memory state, which is much smaller than the memory state in the phase 2) and is transferred to the target edge server s'.

3) Restore application: In this phase, the migrated container and application at target edge server s' is restored. The edge server s' provides service/application for the mobile user.

4) Kill the application on the source edge server: If the previous steps run successfully, the stopped applications on the source node can be killed.

B. The Algorithm Design of PMHM

In the PMHM, we optimize the container migration procedure to optimize the service migration time by pre-migrating a container base image and application data as soon as the target server is obtained by ELMP, which are described from Line 4 to Line 6 in Algorithm 2. Algorithm 2 describes the procedure on migration module of source edge server s, Algorithm 3 describes the procedure on migration module of edge server s', and Algorithm 4 describes the procedure on planner module.

Algorithm 2 The Procedure on Planner Module of the Controller

Require:the controller has received the instruction about migration

Input parameters: the source edge server s, the target edge server s', Suc=0

1: **if** $P_{s'} < P_{min}$ or $P_s < P_{min}$ **then**
2: **return** Error;
3: **end if**
4: Send container creation instruction to s'
5: Set the new container as C'
6: Copy container base image and application image to s'
7: **if** $success(C')$ **then**
8: Clone the base image and application on new container C';
9: Send memory state migration instruction to s;
10: **end if**
11: **if** $(success(s))$ **then**
12: Implement Handover between b and b';
13: **end if**
14: Suc $= 1$
15: **return** Suc

Algorithm 3 The Procedure on Migration Module of Source Edge Server s

Require: the controller have sent the memory state migration instruction to edge server s

Input parameters: the source edge server s, the target edge server s', Suc=0

1: **if** $P_s < P_{min}$ **then**
2: **return** Error;
3: **end if**
4: **if** receive the memory state migration instruction **then** then
5: Pre-dump the container C_u that hosts the ongoing applications for mobile user u, Save the file as f;
6: Transfer the file f to edge server s';
7: Dump the container C_u and Save delta memory state changes as the dumped file as f_d;
8: Stop the container C_u;
9: Transfer f_d to edge server s';
10: Send success message to the controller $success(s)$
11: **end if**

C. Service Migration Time Analysis

The service migration time starts from the time that the source and target edge servers receive the migration instructions, to the time that the service on the target edge server have resumed. In the proposed PMHM, the service migration time include memory state migration time T_{mem}, delta memory state migration time T_{del}, and the resume time T_{res}. The service migration time is computed as follows:

$$T_{total} = T_{mem} + T_{del} + T_{res}, \tag{21}$$

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:08:44 UTC from IEEE Xplore. Restrictions apply.

Algorithm 4 The Procedure on Migration Module of Edge Server s'

Require: the controller have sent a memory state migration instruction to edge server s'

if $P_{s'} < P_{min}$ **then**
  return Error;
end if
if receive a container creation instruction from controller **then**
  Create new container C';
  Send creation success message about C' to controller;
  Create access address $A^{C'}$ of container C';
  Send $A^{C'}$ to edge server s;
  Receive the pre-dumped file f from edge server s;
  Receive the delta memory state file f_d;
  if exist f and f_d **then**
    Resume the migrated applications;
  end if
  Send success message $success(s')$ to the controller
end if

If the target edge server only has basic container image but no application image, the time for copying application image data is required to add into the service migration time. Service migration time is computed as follows:

$$T_{total} = T_{app} + T_{mem} + T_{del} + T_{res}, \tag{22}$$

If the target edge server does not have basic container image as well as the application image, the disk migration time include the time for copying base image and application image, which is denoted as $T_{base} + T_{app}$. Service migration time is computed as follows:

$$T_{total} = T_{base} + T_{app} + T_{mem} + T_{del} + T_{res}, \tag{23}$$

T_{mem} includes checkpoint time and transfer time. The checkpoint time refers to the checkpoint time, which depends on the container image size and computation power of the source edge server s. The transfer time refers to the time to transfer memory state migrations checkpointed files from the source edge server to the target edge server, which depends on the size of memory state files and the bandwidth between the source edge server and the target edge server. Therefore, T_{mem} can be inferred by using the parameters mentioned above.

T_{del} is the time to transfer delta memory state files, which depends on the size of delta memory state files f^d and the bandwidth between the source edge server and the target edge server. The size of delta memory state can be computed using the method as follows. When the container processes the end users' tasks for a certain period of time, the hosting edge server triggers two consecutive checkpoints while leave the container running. Then the server measures the size of the two checkpointed files, where the difference between the two is the delta memory state files size.

T_{res} is the time for resuming the migrated container application at the target edge server, which depends on the computation power of the target edge server and can be inferred by using the historical information of resuming the container application.

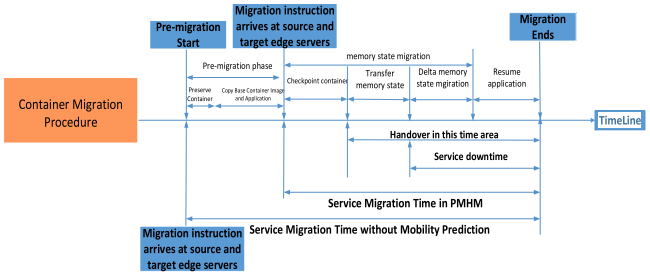


Fig. 3. PMHM Timeline.

T_{ho} refers to the time to handover from the BS b to BS $b_{\prime}$, which can be computed using the relative RSSI hysteresis technique.

Based on the above analysis, the service migration time can be estimated. We explain why the PMHM reduce the service migration time comparing with the method without the mobility prediction. In the PMHM, when the mobile user u moves from the BS b to BS b', the container image and application data have been copied on the target edge server by using mobility prediction. That is to say, the container base image and application image have been available by finishing pre-migration phase. When the source and target edge server receive the migration instruction from the controller, only memory state migration is implemented, service migration time is computed based on the equation (21). If mobility prediction is not performed, pre-migration cannot be performed. Meanwhile, the target edge server may not have container basic images or application data. In this condition, when the source and target edge servers receive migration instructions, they need to migrate not only the container basic images or application image files, but also the memory state migration, that is, equation (22) or equation (23) is utilized to calculate the service migration time. Compared with Formula (22) and Formula (23), Formula (21) obviously has the smallest service migration time. Especially for applications with large container base images and application file data, PMHM will significantly reduce the service migration time. As shown in Fig. 3, the timeline describes the container migration procedure described earlier. And it shows that the service migration time with or without mobility prediction. Furthermore the timeline presents the time plan to perform pre-migration, memory state migration, delta memory state migration trigger handover, and resume application.

VII. Experiments Results

We evaluate our ELMP using a real-world dataset collected form location based social networks. We first describe the dataset. Then, we compare ELMP with the following methods and analyzes the performance of ELMP.

(1) RNN [27]: RNN is one of the basic learners used in this paper. This method takes into account the sparsity of user mobility, spatio-temporal regularity, and the adopted neural network model uses spatio-temporal distance.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:08:44 UTC from IEEE Xplore. Restrictions apply.

TABLE I
STATISTICS OF THE DATASETS

Dataset	Gowalla
Users	52979
POIs	121851
Checkins	3300986
Collection period	2009.02—2010.10

(2) PEMC: PEMC is also one of the basic learners used in this paper. This method embeds users' personalized preferences in the Markov chain model, taking into account the transition of location sequence and geographical preferences.

(3) AMM [42]: This method introduces an attention mechanism based on user context information in the Markov chain model. Here, the time of record generation is taken as the context information, and the time interval between two consecutive records is considered as the factor for setting the sequence transition probability.

A. Dataset

We evaluate ELMP based on the dataset announced by Gowalla (http://snap.stanford.edu/data/loc-Gowalla.html). Gowalla is a location-based social networking website where users share their locations by checking-in. Every time a mobile user creates use the Gowalla, a check-in is created. This dataset includes location-related social networking website where users share their locations by checking-in. The dataset contained 52,979 check-ins of 3,300,986 users over the period of Feb. 2009 - Oct. 2010. Table I shows the statistics of the datasets. We chronologically split all the mobility traces into 80% for training and 20% for testing. In Gowalla dataset, a mobility record contains user identifier (userid), checkin time (timestamp), latitude, longitude, location identifier (locationid). To observe the mobility traces of end users, we randomly selected a batch of mobility record of twelve users from this anonymous data set.

Due to the sparsity of user check-ins datasets, a sing users checkin records cannot well characterize a users temporal regularity. For example, if we consider weekly mobility traces with hour granularity (i.e., 168 h in a week), we found that there were much time for each user that we did not observe any mobility behavior. Therefore, we select mobility records of three group of end users for observation. From the spatial perspective, we observe clearly the mobility traces of a user limited in certain geographic areas and the locations the users frequently visited, as plotted in the Fig. 4. Such observation indicates that mobility behaviors have strong geographic preference and different users usually have their own frequented visited locations. Fig. 5 depicts the location distribution of all users at a specific time, which show the location where the user stays between 21:00 and 22:00 (marked by five pointed), and the location where he stays between 16:00 and 17:00 (marked by plus sign). Based on the above observation, we can see that user mobility has spatiotemporal regularity and aggregation. The above characteristics motivate us to design proactive edge service migration strategies based on mobility prediction.

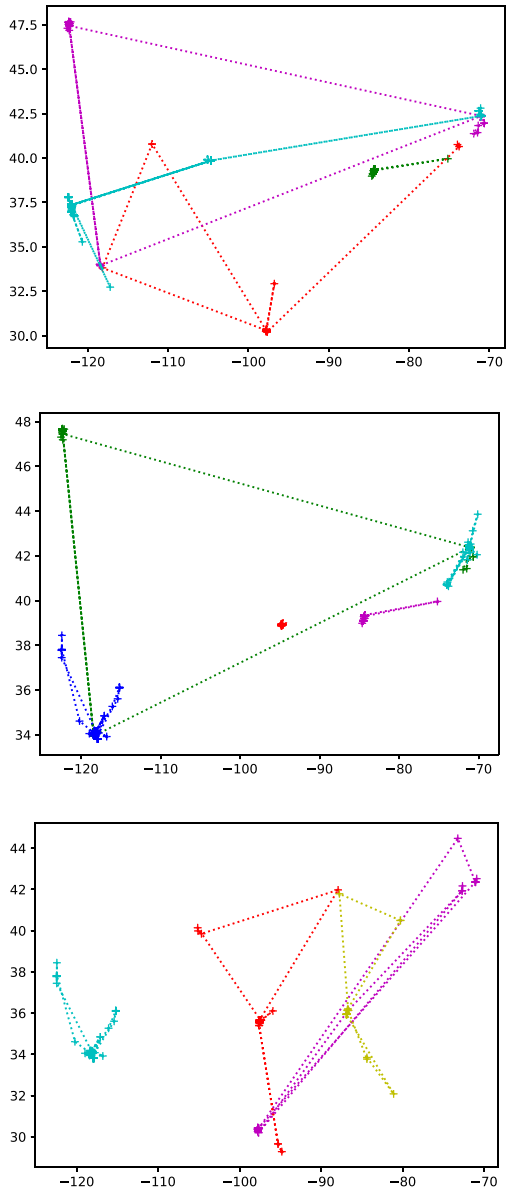


Fig. 4. The mobility traces of end users randomly selected.

B. Performance of the ELMP

We trained the proposed user mobility prediction approach ELMP using the parameters in Table II. Data processing is described in Section V-D. The ensemble parameter w_1, w_2 and the order of the GEMC predictor m are independently adjusted during the training processes.

(1) Accuracy: We evaluate the performance of the ELMP using $acc@l$, which can be defined as $acc@l = num(hit@l)/num(tests)$. We rank the candidate locations by the probabilities generated from the model ELMP, and check whether the ground-truth location appears in the top l candidate edge servers. The number of the records appearing in the top l candidate edge servers is denoted as $num(hit@l)$. The number of total testing samples is denoted as $num(tests)$.

In Fig. 6, we show the results grouped based on the number of neighbors l in their candidate edge servers when $w_1 = 0.8$, $w_2 = 0.2$, and $m = 5$. For comparison, we do report the

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:08:44 UTC from IEEE Xplore. Restrictions apply.

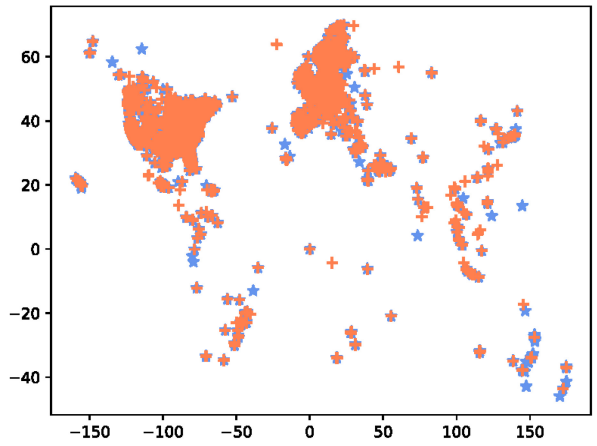


Fig. 5. The location of end users at the specific time of the day, i.e., 21:00 and 22:00 (colored in blue),16:00 and 17:00 (colored in orange).

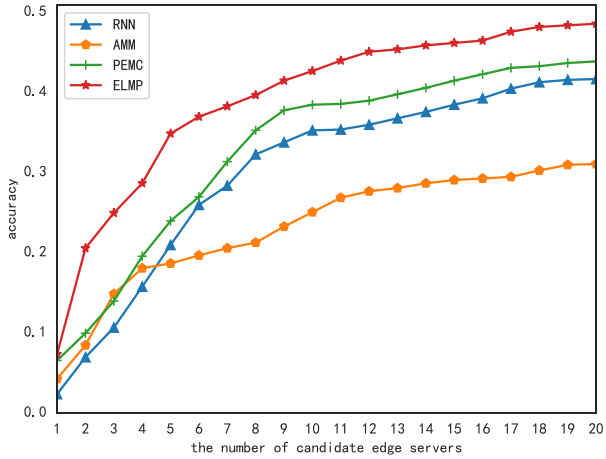


Fig. 6. The accuracy using different number of candidate edge servers.

TABLE II
PARAMETER SETTINGS

Parameter	Value
batch size, $batch_size$	600
learning rate, $learning_rate$	0.01
the epochs of training, $epochs$	100
the number of hiddenlayers, $hiddenlayer$	10
the decay rate of weights, $weightdecay$	0.01
the epoch of validation, $validateepoch$	5
the time interval threshold between two adjacent check-ins, τ	6 hours
the weight of different kinds of factors influencing user mobility, γ	0.2
the temporal decay rate, α	0.1
the spatial decay rate, β	1000

cases when the candidate set contains less than twenty edge servers, because they rarely occurred in our simulations when the number of neighbors l in their handover set is more than twenty, and hence we could not assess the training algorithm in such cases.

Compared with RNN, PEMC, and AMM, the prediction accuracy of ELMP is higher than the other three benchmark methods. ELMP is 6 percentage points higher than PEMC,15 percentage points higher than AMM, and 9 percentage points higher than RNN.

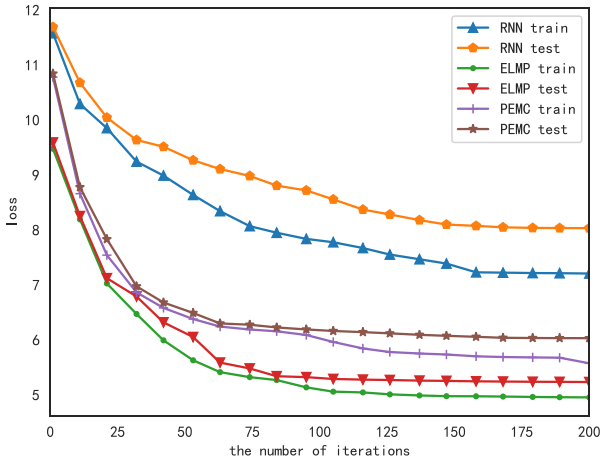


Fig. 7. The loss using different number of iterations.

With the increase of the number of candidate edge servers, the prediction accuracy gradually tends to be stable. Compared with the benchmark methods, PEMC and RNN consider the spatio-temporal factors, the prediction performance of PEMC and RNN exceeds AMM in most cases, because AMM only considers the time of two consecutive location records when setting the transition probability.

As can be seen from the above simulation results, ELMP can capture complex mobile behaviors from the sequence of previously visited edge servers and predict the next edge server with higher accuracy.

(2) Loss: the cross-entropy loss is used to evaluate the convergence performance of ELMP. Fig. 7 shows the cross-entropy loss values of different methods on the training set and test set. It can be seen that ELMP has the smallest loss value compared with other benchmark methods. ELMP has better convergence, proving that the proposed ELMP method has better adaptability for user mobility.

In conclusion, the ELMP based on ensemble learning takes into account the time interval between two consecutive location records, the geospatial distance, personal preferences and other factors. And the accuracy and convergence are significantly higher than other benchmark methods. Therefore, ELMP has better prediction performance and adaptability for user mobility.

C. The Performance of PMHM

We simulate our proposed PMHM solution using our testbed, which includes Pycharm and Python 3.6. The bandwidth between any two adjacent edge servers is configured as 100 Mbps or 300 Mbps. The bandwidth between a mobile user and edge server is configured as 100Mbps or 200Mbps. The bandwidth between the edge server and central cloud edge server is configured as 50Mbps.

In this paper, three applications are adopted and packaged into Docker containers. The Yolo-based object recognition service has a file system size of 792MB [18], the Busybox container has a file system size of 290KB [44], and the Openface-based service has a file system size of 1.86GB [45]. This paper contrasts PMHM with the following methods:

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:08:44 UTC from IEEE Xplore. Restrictions apply.

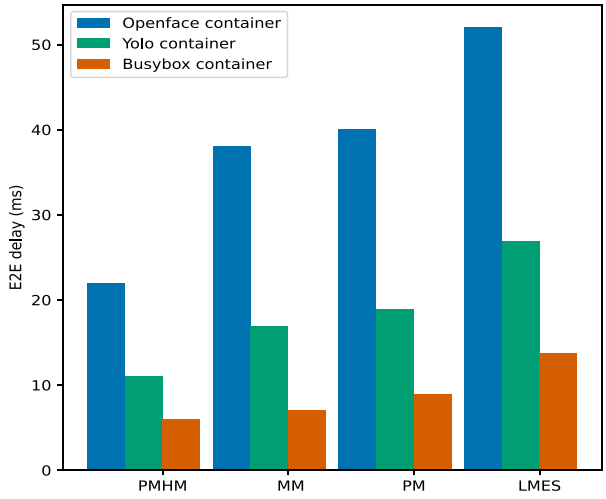


Fig. 8. E2E delay $B_{ss'} = 100Mbps$.

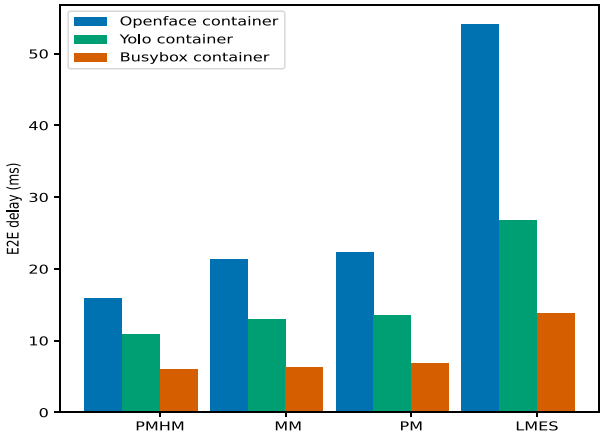


Fig. 9. E2E delay $B_{ss'} = 300Mbps$.

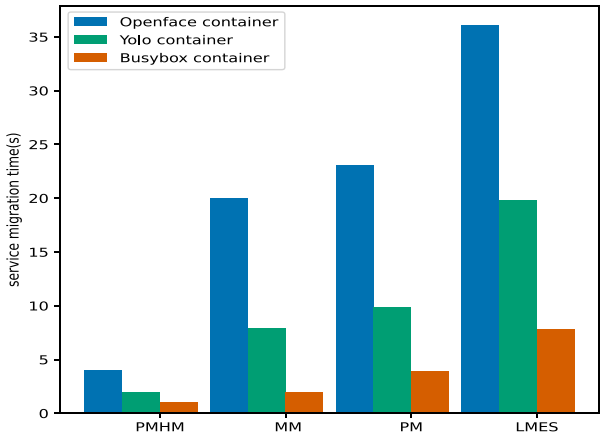


Fig. 10. Total service migration time $B_{ss'} = 100Mbps$.

1) The method PM (Partial Migration, PM) proposed in literature [39]: Partial migration is one of the proposed migration methods under the predefined mobility model. Basic images and applications are replicated to target edge servers after the mobile user moves.

2) The method MM (Memory Migration, MM) proposed in the literature [39]: MM is also the proposed migration method in the paper [39] under the predefined mobile mode, in which the basic image file on the target server is available. After the mobile user moves, the application is replicated from the source server to the target edge server.

3) The method LMES (Live Migration of Edge Service, LMES) proposed in the literature [44]: After the mobile user moves, the basic image file and application are downloaded from the centralized cloud server to the nearest and nearby potential target edge servers.

The performance of the PMHM is affected by the following factors, including the network bandwidth between the source edge server and the target edge server, the available resources allocated by the edge servers, such as the computation speed, and the network bandwidth between the mobile user and the edge server. Considering the above factors, different network bandwidth are used to evaluate the efficiency of service migration. Here $B^t_{ubs} = 50Mbps$, $B^{t'}_{ub's'} = 100Mbps$. P_s and $P_{s'}$ are configured as 1000Cycles/s and 3000Cycles/s. We evaluate the above MEC system in terms of E2E delay and the total migration time.

1) E2E delay analysis: we give the results of E2E delay under the PMHM and three other benchmark methods in Fig. 8 and Fig. 9. As shown in Fig. 8, the bandwidth between the source edge server and the target edge server is set to 100 Mbps. For Busybox, the end-to-end delay under PMHM is at least 16.7% lower than that of MM, PM, and LEMS. For Yolo, the end-to-end delay under PMHM is at least 54.1% lower than that of MM, PM, and LEMS. For Openface, the end-to-end delay under PMHM is at least 54.1% lower than that of MM, PM, and LEMS; The end-to-end delay is at least 71.7% lower than MM, PM, and LEMS. From the above results, it can be seen that the method proposed in this paper is more beneficial for applications with larger container base images and application files.

As shown in Fig. 9, the bandwidth between the source edge server and the target edge server is increased and set to 300Mbps. The E2E latency under PMHM, MM, and PM is roughly the same for Busybox and Yolo. This is because the base images and application files of the Busybox and Yolo containers are small, and the large bandwidth between the source edge server and the target edge server has little effect on reducing latency. However, for Openface with the largest file size, the end-to-end latency under PMHM is at least 33% lower than that under MM, PM, and LEMS. If the basic image file and application copy are downloaded from the central cloud to the nearest target edge server, the file needs to be transferred through the WAN. Therefore, the end-to-end delay under LMES is the highest. Overall, PMHM has the lowest E2E latency compared to the other three methods.

2) Total service migration time analysis: Different migration schemes PMHM, MM, PM, and LEMS are represented on the X-axis, while the Y-axis represents time in seconds. In Fig. 10 and Fig. 11, the bandwidth between the source edge server and destination edge server is set to 100Mbps and 200Mbps.

As shown in Fig. 10, compared with MM, PM, and LEMS, PMHM has the shortest service migration time because there is

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:08:44 UTC from IEEE Xplore. Restrictions apply.

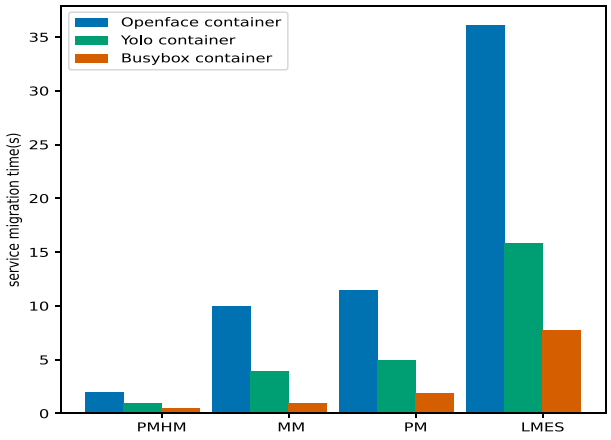


Fig. 11. Total service migration time $B_{ss'} = 200Mbps$.

no need to migrate base container image copies and application copies during service migration, which have been predicted by mobility prediction. It can be observed that for MM and PM, the long migration time is due to the copying of the base image. For LEMS, the long migration time is due to the copying for the application. In Fig. 11, the bandwidth between the source edge server and the target edge server is set to 200Mbps. The service migration time under PMHM is significantly lower than the other three schemes, and the performance of MM and PM is about the same. Due to the low bandwidth of WLAN, the service migration time under LEMS is still the highest. Therefore, PMHM is a more promising solution in the case of low bandwidth and large file system applications, because base container image replication and application replication are available before in-memory state migration, and only memory state files are copied.

VIII. Conclusion

In this work, MAPSM is proposed for MEC. Our approach incorporate mobility prediction with service migration procedure. The mobility prediction in ELMP is expressed as a probability vector over the neighboring edge server, which is the next access point of the offloading service. The ELMP combines recurrent neural networks and geographical embedding Markov chain based on soft weighted average ensemble method, is trained and exploited in Mobility Prediction module at the controller. Then the prediction result are utilized as the input of PMHM in Planner module. PMHM exploits container base image copy, application copy, delta memory state copy, together with communication handover, optimizing total migration time and end to end delay. Simulation results demonstrate the effectiveness of our proposed solution in comparison with the benchmark migration strategies. Our solution effectively reduces the total migration time and end to end delay by pre-migration and communication handover. In the future work, we will extend the proposed approach to the cases where 1) the migrated service involves multiple end users and containers 2) the hardware resource on the target edge server are constraint.

References

[1] H. Tataria, M. Shafi, A. F. Molisch, M. Dohler, H. Sjöland, and F. Tufvesson, "6G wireless systems: Vision, requirements, challenges, insights, and opportunities," *Proc. IEEE*, vol. 109, no. 7, pp. 1166–1199, Jul. 2021.

[2] K. H. K. Reddy, R. S. Goswami, A. K. Luhach, P. Chatterjee, M. Alnumay, and D. S. Roy, "EFLSM: An intelligent resource manager for fog layer service management in smart cities," *IEEE Trans. Consum. Electron.*, vol. 70, no. 1, pp. 2281–2289, Feb. 2024.

[3] W. Li and J. Guo, "Adaptive migration method for virtual network functionality in cloud and blockchain-based integrated environment for handling socio-economic systems," *IEEE Trans. Consum. Electron.*, vol. 70, no. 4, pp. 6952–6964, Nov. 2024.

[4] S. Wang, R. Urgaonkar, M. Zafer, T. He, K. Chan, and K. K. Leung, "Dynamic service migration in mobile edge computing based on Markov decision process," *IEEE/ACM Trans. Netw.*, vol. 27, no. 3, pp. 1272–1288, Jun. 2019.

[5] J. Li, D. Zhao, Z. Shi, L. Meng, W. Gaaloul, and Z. Zhou, "Energy-efficient online service migration in edge networks," *IEEE Internet Things J.*, vol. 11, no. 18, pp. 29689–29708, Sep. 2024.

[6] X. Li, S. Chen, Y. Zhou, J. Chen, and G. Feng, "Intelligent service migration based on hidden state inference for mobile edge computing," *IEEE Trans. Cogn. Commun. Netw.*, vol. 8, no. 1, pp. 380–393, Mar. 2022.

[7] S. Yu, H. Chen, and H. Jin, "Nereus: A distributed stream band join system with adaptive range partitioning," *IEEE Trans. Consum. Electron.*, vol. 69, no. 4, pp. 949–961, Nov. 2023.

[8] L. Liang, J. Xiao, Z. Ren, Z. Chen, and Y. Jia, "Particle swarm based service migration scheme in the edge computing environment," *IEEE Access*, vol. 8, pp. 45596–45606, 2020.

[9] B. Cao, H. Ye, J. Liu, B. Tang, Z. Tao, and S. Deng, "SMART: Cost-aware service migration path selection based on deep reinforcement learning," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 9, pp. 12421–12436, Sep. 2024.

[10] Z. Liang, Y. Liu, T. M. Lok, and K. Huang, "Service migration for multi-cell mobile edge computing," in *Proc. IEEE Global Commun. Conf.*, 2020, pp. 1–6.

[11] J. Li, W. Liang, M. Chen, and Z. Xu, "Mobility-aware dynamic service placement in D2D-assisted MEC environments," in *Proc. IEEE Wireless Commun. Netw. Conf. (WCNC)*, 2021, pp. 1–6.

[12] X. Chen et al., "Dynamic service migration and request routing for microservice in multicell mobile-edge computing," *IEEE Internet Things J.*, vol. 9, no. 15, pp. 13126–13143, Aug. 2022.

[13] Y. Chen, Y. Sun, C. Wang, and T. Taleb, "Dynamic task allocation and service migration in edge-cloud IoT system based on deep reinforcement learning," *IEEE Internet Things J.*, vol. 9, no. 18, pp. 16742–16757, Sep. 2022.

[14] F. Guo and M. Peng, "Efficient mobility management in mobile edge computing networks: Joint handover and service migration," *IEEE Internet Things J.*, vol. 10, no. 20, pp. 18237–18247, Oct. 2023

[15] Q. Cao, "Service migration strategy considering task deadline in edge computing," in *Proc. Asia-Pacific Conf. Image Process., Electron. Comput. (IPEC)*, Dalian, China, 2023, pp. 453–461.

[16] H. Wei, H. Luo, and Y. Sun, "Mobility-aware service caching in mobile edge computing for Internet of Things," *Sensors*, vol. 20, no. 3, pp. 1–20, Jan. 2020.

[17] F. Lyu, J. Ren, P. Yang, N. Cheng, Y. Zhang, and X. S. Shen, "SoSA: Socializing static APs for edge resource pooling in large-scale WiFi system," in *Proc. 39th IEEE Conf. Comput. Commun. (INFOCOM)*, Aug. 2020, pp. 1181–1190.

[18] M. V. Ngo, T. Luo, H. T. Hoang, and T. Q. S. Ouek, "Coordinated container migration and base station handover in mobile edge computing," in *Proc. IEEE Global Commun. Conf.*, 2020, pp. 1–6.

[19] Z. Liang, Y. Liu, T. M. Lok, and K. Huang, "Multi-cell mobile edge computing: Joint service migration and resource allocation," *IEEE Trans. Wireless Commun.*, vol. 20, no. 9, pp. 5898–5912, Sep. 2021.

[20] I. Labriji et al., "Mobility aware and dynamic migration of MEC services for the Internet of Vehicles," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 1, pp. 570–584, Mar. 2021.

[21] S. Ghosh, A. Mukherjee, S. K. Ghosh, and R. Buyya, "Mobi-IoST: Mobility-aware cloud-fog-edge-IoT collaborative framework for time-critical applications," *IEEE Trans. Netw. Sci. Eng.*, vol. 7, no. 4, pp. 2271–2285, Oct.–Dec. 2020.

[22] A. G. Joseph, M. M. Gokhale, J. M. J. Mani, and T. Veni, "Mobility aware computation offloading in fog devices using virtual machine migration," in *Proc. 14th Int. Conf. Comput. Commun. Netw. Technol. (ICCCNT)*, 2023, pp. 1–6.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:08:44 UTC from IEEE Xplore. Restrictions apply.

[23] S. Raj, N. Mangal, S. Savitha, and S. Salvi, "Virtual machine migration in heterogeneous clouds—A practical approach," in *Proc. IEEE Int. Conf. Electron., Comput. Commun. Technol. (CONECCT)*, Bengaluru, India, 2020, pp. 1–6.

[24] W. Lu, X. Meng, and G. Guo, "Fast service migration method based on virtual machine technology for MEC," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4344–4354, Jun. 2019.

[25] K. Hasan and S. H. Jeong, "An LSTM-based mobility prediction mechanism in the ICN-based vehicular networks," in *Proc. 27th Asia–Pacific Conf. Commun. (APCC)*, Jeju Island, Republic of Korea, 2022, pp. 244–246.

[26] Y. Chen, C. Long, G. Cong, and C. Li, "Context-aware deep model for joint mobility and time prediction," in *Proc. 13th Int. Conf. Web Search Data Min.*, 2020, pp. 106–114.

[27] D. Yang, B. Fankhauser, P. Rosso, and P. Cudre-Mauroux. "Location prediction over sparse user mobility traces using RNNs: Flashback in hidden states!," in *Proc. 29th Int. Joint Conf. Artif. Intell.*, pp. 2184–2190.

[28] P. Zhang, H. N. Koutsopoulos, and Z. Ma, "DeepTrip: A deep learning model for the individual next trip prediction with arbitrary prediction times," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 6, pp. 5842–5855, Jun. 2023.

[29] S. Siham, I. Nisrine, B. Hanae, and M. Khalid, "Exploring the effectiveness of neural network algorithms on mobility patterns: A comprehensive metrics-based study," in *Proc. IEEE 12th Int. Symp. Signal, Image, Video Commun. (ISIVC)*, Marrakech, Morocco, 2024, pp. 1–6.

[30] X. Dong, Z. Yu, W. Cao, Y. Shi, and Q. Ma, "A survey on ensemble learning," *Front. Comput. Sci.*, vol. 14, no. 2, pp. 241–258, 2020.

[31] F. Schwenker, "Ensemble methods: Foundations and algorithms [book review]," *IEEE Comput. Intell. Mag.*, vol. 8, no. 1, pp. 77–79, Feb. 2013.

[32] T. G. Dietterich, "Ensemble methods in machine learning," in *Proc Int. Workshop Multiple Classifier Syst.*, 2000, pp. 1–15.

[33] H. Wang, Y. Yu, Y. Cai, X. Chen, L. Chen, and Y. Li, "Soft-weighted-average ensemble vehicle detection method based on single-stage and two-stage deep learning models," *IEEE Trans. Intell. Veh.*, vol. 6, no. 1, pp. 100–109, Mar. 2021.

[34] B. Shi, H. Shen, B. Dong, and Q. Zheng, "Memory/disk operation aware lightweight VM live migration," *IEEE/ACM Trans. Netw.*, vol. 30, no. 4, pp. 1895–1910, Aug. 2022.

[35] Z. Ma, D. Ma, M. Lv, and Y. Liu, "Virtual machine migration techniques for optimizing energy consumption in cloud data centers," *IEEE Access*, vol. 11, pp. 86739–86753, 2023.

[36] S. Choudhury, S. Maheshwari, I. Seskar, and D. Raychaudhuri, "ShareOn: Shared resource dynamic container migration framework for real-time support in mobile edge clouds," *IEEE Access*, vol. 10, pp. 66045–66060, 2022.

[37] X. Xie, X. Wu, Q. Hu, and S. Yuan, "Multi-container migration strategy optimization for industrial robotics workflow based on hybrid tabu-evolutionary algorithm," *IEEE Trans. Services Comput.*, vol. 17, no. 5, pp. 2640–2653, Sep./Oct. 2024.

[38] P. Bellavista, A. Corradi, L. Foschini, and D. Scotece, "Differentiated service/data migration for edge services leveraging container characteristics," *IEEE Access*, vol. 7, pp. 139746–139758, 2019.

[39] R. A. Addad, D. L. C. Dutra, M. Bagaa, T. Taleb, and H. Flinck, "Fast service migration in 5G trends and scenarios," *IEEE Netw.*, vol. 34, no. 2, pp. 92–98, Mar./Apr. 2020.

[40] S. Mallick. "OpenCV YOLOv3 with CPU." Accessed: Aug. 2, 2020. [Online]. Available: https://www.learnopencv.com/tag/yolov3/

[41] S. Feng, X. Li, Y. Zeng, G. Cong, Y. M. Chee, and Q. Yuan, "Personalized ranking metric embedding for next new poi recommendation," in *Proc. 24th Int. Joint Conf. Artif. Intell.*, Jun. 2015, pp. 2069–2075.

[42] H. Wang, Y. Li, D. Jin, and Z. Han, "Attentional Markov model for human mobility prediction," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 7, pp. 2213–2225, Jul. 2021.

[43] "Arm: The route to a trillion devices." Accessed: Mar. 1, 2020. [Online]. Available: http://pages.arm.com/route-to-trillion-devices.html.

[44] L. Ma, S. Yi, N. Carter, and Q. Li, "Efficient live migration of edge services leveraging container layered storage," *IEEE Trans. Mobile Comput.*, vol. 18, no. 9, pp. 2020–2033, Sep. 2019.

[45] S. Mallick. "OpenCV YOLOv3 with CPU [EB/OL]." [Online]. Available: https://www.learnopencv.com/tag/yolov3/,2020-8-2.

Xuhui Zhao received the master's degree from Zhengzhou University, China, and the Ph.D. degree from the Beijing University of Posts and Telecommunications in 2022. She is an Associate Professor with the Computer department, College of Information Engineering, Henan University of Science and Technology, China, and also with State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications. Her research interests include mobile edge computing, artificial intelligence, and vehicular networks.

Yan Shi received the Ph.D. degree from the Beijing University of Posts and Telecommunications in 2007, where she is currently an Associate Professor with the School of Computer Science (National Pilot Software Engineering School). Her current research interests include vehicular networks and mobile computing, especially mobility management technology.

Shanzhi Chen (Fellow, IEEE) received the Ph.D. degree from the Beijing University of Posts and Telecommunications, China, in 1997. He joined the Datang Telecom Technology and Industry Group and the China Academy of Telecommunication Technology (CATT) in 1994, and has been serving as the EVP of Research and Development since 2008. He is currently the Director of the State Key Laboratory of Wireless Mobile Communications, CATT, where he conducted research and standardization on 4G TD-LTE and 5G. His current research interests include 5G mobile communications, network architectures, vehicular communication networks, and Internet of Things. His achievements have received multiple top awards and honors by the China central government, especially the Grand Prize of the National Award for Scientific and Technological Progress, China, in 2016 (the highest Prize in China). He has served as an Area Editor for IEEE INTERNET OF THINGS JOURNAL, an Editor for IEEE NETWORK, and a Guest Editor for IEEE WIRELESS COMMUNICATIONS, the *IEEE Communications Magazine*, and IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY, as well as a member of TPC Chairs of many international conferences.

Jianghui Liu was born in Xinxiang, China, in January 1982. He received the bachelor's degree in computer science and technology and the master's degree in control engineering from the Henan University of Science and Technology in 2005 and 2009, respectively. He currently works with the Henan University of Science and Technology. His research interests include distributed optimization and mobile-edge computing.

Baofeng Ji received the Ph.D. degree in information and communication engineering from Southeast University, China, in 2014, where he has been a Postdoctoral Fellow with the School of Information Science and Engineering since 2014. He is also a Professor with the Henan University of Science and Technology. He has published over 40 peer-reviewed papers, published three scholarly books, holds six invention patents, and has submitted six technical contributions to IEEE standards. His current research interests include MIMO wireless communications, cooperative wireless communications, and millimeter-wave wireless communications.

Shahid Mumtaz (Senior Member, IEEE) is currently with the Department of Applied Informatics, Silesia University of Technology and the Department of Engineering, Nottingham Trent University. He has more than 12 years of wireless industry experience. His research interests include architectural enhancements to 3GPP networks, 5G NR-related technologies, green communications, cognitive radio, cooperative networking, radio resource management, network slicing, LAA/LTU, cross-layer design, backhaul/fronthaul, heterogeneous networks, M2M and D2D communication, and baseband digital signal processing.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:08:44 UTC from IEEE Xplore. Restrictions apply.